

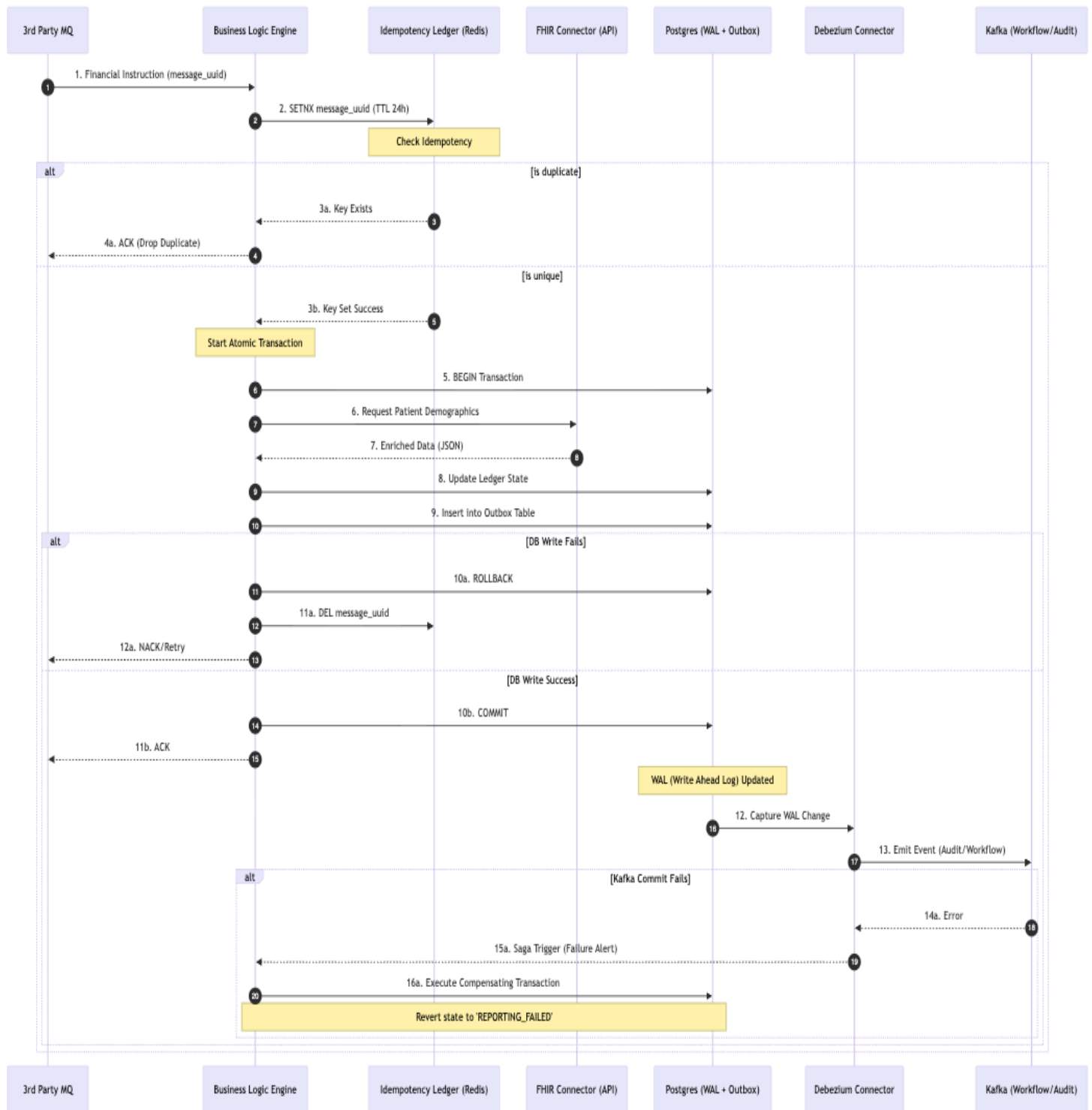
Atomic Financial Settlement Flow (The "Jackpot Cycle")

1. System Overview

The engine is designed to ingest financial instructions from an external MQ, perform idempotent validation, enrich data via a FHIR (Fast Healthcare Interoperability Resources) API, and persist state changes to a PostgreSQL ledger. To ensure system-wide consistency without the overhead of 2PC (Two-Phase Commit), the system utilizes **Change Data Capture (CDC)** via Debezium to stream audit events to Kafka

2. Technical Component Stack

Component	Technology	Responsibility
Ingress	3rd Party MQ (RabbitMQ/IBMMQ)	Source of financial instructions.
Idempotency	Redis	Atomic lock and \$24h\$ message_uuid tracking.
Database	PostgreSQL 15+	System of record; utilizes WAL for CDC.
Enrichment	FHIR Connector	RESTful API call for patient demographics.
Event Stream	Kafka + Debezium	Reliable event delivery and audit logging.



3. Detailed Data Flow & Logic

1. **Ingress:** The Logic Engine consumes a message. It extracts the `message_uuid`.
2. **Idempotency Check:** The engine executes `SETNX <message_uuid> 1 EX 86400` in Redis.
 - If result = `0`: Drop message (Duplicate).
 - If result = `1`: Proceed.
3. **Atomic Transaction Block:**
 - **Start Transaction:** Open a standard SQL transaction.
 - **FHIR Call:** Fetch patient data. **Timeout must be set to \$50ms\$** to preserve latency targets.
 - **State Change:** Update Ledger Table with `status = 'SETTLED'` and FHIR enrichment data.
 - **Outbox Write:** Insert the event payload into the `outbox_events` table within the *same* transaction.
4. **Commit/Rollback:**
 - On success: `COMMIT`. The WAL is updated.
 - On FHIR/DB failure: `ROLLBACK`. Redis key is deleted to allow for a retry.
5. **CDC Emission:** Debezium monitors the WAL/Outbox. It picks up the commit and pushes to Kafka.
6. **Saga Recovery:** If Kafka rejects the message (e.g., broker down), the Saga Coordinator triggers a compensating transaction to mark the DB record as `SYNC_PENDING` or `FAILED_AUDIT`.

4. Engineering Constraints (NFRs)

- **Latency (\$T\$):** Total duration from MQ receipt to Kafka emission must be $T < 200ms$.
- **Isolation Level:** PostgreSQL transaction isolation must be set to `Read Committed` at minimum, `Repeatable Read` preferred.
- **Retry Policy:** FHIR connector must implement a "Fail-Fast" strategy (0 retries inside the DB transaction).

5. QA Test Scenarios (Manual/Functional)

Scenario A: Idempotency Validation (24h Window)

- **Action:** Submit Message A. Wait 10 seconds. Submit Message A again with identical `message_uuid`.
- **Expected Result:** Second message is ignored by the logic engine; Redis log shows key exists; DB contains exactly one entry.
- **Action:** Submit Message B. Wait 24 hours + 1 minute. Submit Message B again.
- **Expected Result:** Message B is processed as a new transaction (Redis TTL expired).

Scenario B: Atomicity & Rollback (FHIR Failure)

- **Action:** Mock the FHIR API to return a `500 Internal Server Error` or a timeout.
- **Expected Result:** DB transaction rolls back. Ledger remains unchanged. Redis key for `message_uuid` is deleted (allowing for manual re-injection).

Scenario C: CDC Integrity & Eventual Consistency

- **Action:** Complete a successful settlement. Query the Postgres `ledger` table and the Kafka `audit_topic`.
- **Expected Result:** Compare JSON fields. 100% field parity (including enriched FHIR data) between DB and Kafka event.

Scenario D: High-Concurrency Throughput

- **Action:** Inject 500 messages with unique IDs simultaneously via MQ.

- **Expected Result:** System maintains processing speed. Monitor logs for "Deadlock Detected" errors in Postgres. All 500 records must reach Kafka in $< 200\text{ms}$ average.

Scenario E: Saga Pattern (Kafka Outage)

- **Action:** Temporarily disconnect the Kafka Producer/Debezium. Process a message.
 - **Expected Result:** DB transaction commits (State: Settled). After Debezium retry limit is hit, the Saga Coordinator must update the DB record to a "Warning" state for manual audit.
-